

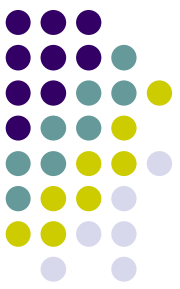
Intro Design Patterns

Architecture of Computer Games
SE 456

Ed Keenan

5 January 2022

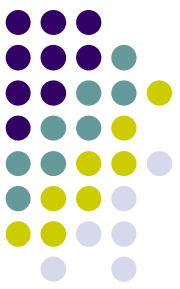
13.0.9.3.2 Mayan Long Count



Overview

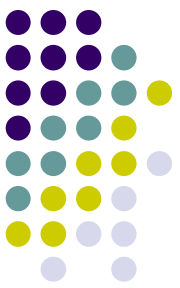
- Design Patterns
 - Intro
 - Need
 - Cohesion / Coupling
 - Benefits
 - Performance
- Patterns
 - Singleton
 - Factory





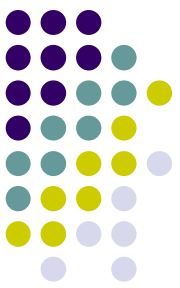
Intro to Design Patterns

- Design Patterns and Object Oriented
 - Intermixed
 - Core Object Oriented principles steer Patterns
- Reference and Links
 - <http://www.uml-diagrams.org/class-reference.html>
 - <https://www.dofactory.com/net/design-patterns>
 - <http://www.oodeesign.com/factory-pattern.html>



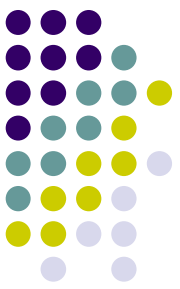
Design Patterns - quick

- Design pattern is a general reusable solution to a commonly occurring problem within a given context in software design.
 - a description or template for how to solve a problem
- Patterns are formalized best practices.
- Show relationships and interactions between classes or objects



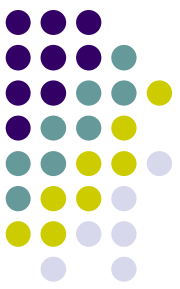
Need for Object-Oriented

- Functional Decomposition is a natural way to deal with complexity.
 - Algorithmic decomposition breaks a process down into well-defined steps
 - Example: appending a string to a file
- Natural side effect
 - Saddles the program with too much responsibility.
 - Does not allow for easy, low risk changes
 - Many bugs originate with changes to code



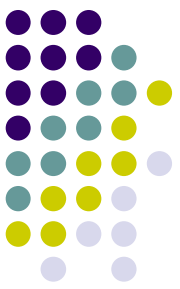
Object-Oriented Paradigm

- Requirements will always change
 - As you understand the problem
 - you will change the requirements
 - Change, change, change!!!
 - Will never stop
- Benefits of Object Orientation
 - Can contain areas of change
 - Insulate code from the effects of change more easily
- Design code so that the impact of changing requirements is much less dramatic.



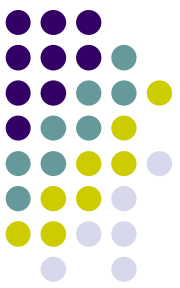
Cohesion / Coupling

- **Cohesion**
 - How closely the operations in a routine are related
- **Coupling**
 - How strongly a routine is related to other routines
- **GOAL**
 - Create routines with internal integrity
 - (strong cohesion)
 - Create routines with small, direct, visible, and flexible relations to other routines
 - (loose coupling)



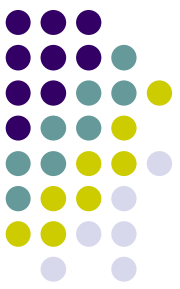
Design Patterns

- Design Patterns and Object-Oriented design reinforce each other.
- Pattern
 - Solution to a problem in a context
 - Describes a core way to solve a problem, such that you can use it over and over again without ever doing it the same way twice
- Patterns exists for almost any design problem
 - May be combined to solve complex problems



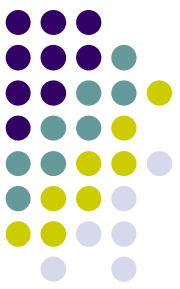
Gang of Four

- ***Design Patterns: Elements of Resusable Object-Oriented Software***
 - Gamma, Helm, Johnson, Vlissides
- Applied the idea of patterns to software design
 - Calling them design patterns



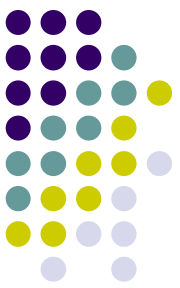
Design Patterns Benefits

- Help with **reuse** and communication
 - Give a higher perspective on analysis and design
- Helps use see the forest and the trees
 - Ability to be external and away from the details
- Improve
 - Communication of Code
 - **Modifiability of Code**
 - Maintainability of Code



Design Patterns Benefits

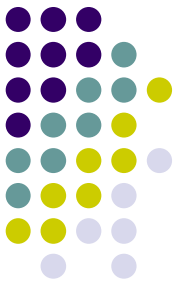
- Illustrate basic Object-oriented principles
 - Design to interfaces
 - Favor Aggregation over inheritance
 - Find what varies and encapsulate it
- Offers alternatives to large inheritance hierarchies



OO vs Performance

- Object-oriented design is sometimes at odds with performance
 - Design standpoint
 - it's best to encapsulate everything
 - treat those objects uniformly
 - reuse them
 - extend them
 - Performance standpoint - we want lean and mean
- ***Thunder Dome***
 - Two man enter one man leaves

Thank You!



- Questions?

